

AMRITA VISHWA VIDYAPEETHAM
SCHOOL OF COMPUTING
DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

Dynamic Evolutionary Optimization Algorithm Switching in AutoML Systems

23CSE399 – Project Phase 1 (Panel Review 2)
March 30, 2026

Team Members		Guide	Project Type	Marks
CB.SC.U4CSE23445	Shivam Kharat	Dr. Thangavelu S. <i>Associate Professor, Dept. of CSE</i>	Type 1 Research	.../ 5
CB.SC.U4CSE23565	Siddharth Ladda			
CB.SC.U4CSE23627	Kaushal Loya			
CB.SC.U4CSE23639	Paarthu Reddy			
CB.SC.U4CSE23642	Priyansh Narang			

1 Introduction & Background

Automated Machine Learning (AutoML) aims to automate the end-to-end process of applying machine learning to real-world problems. A core component of AutoML is Hyperparameter Optimization (HPO), which searches for the optimal configuration of a given model. As datasets and models grow in complexity, the search landscapes become highly non-convex and unpredictable. Consequently, selecting an efficient optimization algorithm becomes crucial. Our motivation stems from the observation that relying on a single optimization strategy often leads to stagnation, whereas dynamically adapting the search strategy can yield superior performance and computational efficiency.

2 Problem Definition & Originality

Problem Statement: Existing AutoML systems rely on a single fixed optimization algorithm (e.g., Genetic Algorithm or Bayesian Optimization) for the entire hyperparameter search. This approach has no mechanism to evaluate whether the chosen algorithm is effective for the current search landscape, cannot switch to a better algorithm if stagnation occurs, and wastes computational resources on underperforming strategies. There is no framework that enables multiple heterogeneous algorithms to compete, cooperate, and switch dynamically during a single optimization run.

Originality Claim:

- **Novel Idea:** We propose a hybrid framework that combines *tiered population management*, *island-based cooperative co-evolution*, and *UCB1 bandit-based algorithm selection* into a unified AutoML optimization engine. Unlike existing approaches where algorithms run independently or are selected statically, our system:
 - Maintains an always-on competition arena where no algorithm is ever permanently eliminated
 - Allocates resources meritocratically — only top-performing algorithms get larger populations
 - Uses reinforcement learning (UCB1) for intelligent, exploration-aware algorithm switching during stagnation
 - Monitors and handles stagnation at multiple levels (individual algorithm, tier-wide, and system-wide)

3 Literature Survey & Gap Analysis

Reference	Approach / Contribution	Limitation identified
XAutoLM [1]	Meta-learning to guide AutoML search.	Relies on a single fixed optimization algorithm throughout the process.
SHSR [2]	Reduces search space via meta-level analysis.	Provides no runtime adaptation or algorithm switching capability.
OnMAR [3]	Introduces an online model-agnostic repair mechanism.	Operates with a single optimizer and cannot leverage multiple algorithms.
MsMA [4]	Uses adaptive meta-level partitioning for stagnation detection.	Stagnation handling not integrated with switching or cooperative frameworks.

Table 1: Summary of Existing Literature and Gap Analysis

Research Gap [5, 6]: No existing system combines dynamic algorithm switching, cooperative multi-algorithm execution, and bandit-based meta-learning into a unified AutoML optimization framework. Current approaches either use fixed algorithms, run multiple algorithms independently without cooperation, or handle stagnation without integrating it into the algorithm selection process.

4 Learning & Fundamentals Completed

Subdomains to Know & Learning Plan:

- **Metaheuristics & Optimization:** Requires a deep understanding of Genetic Algorithms (GA), Particle Swarm Optimization (PSO), Differential Evolution (DE), and CMA-ES.
- **Reinforcement Learning (RL):** Focuses on Multi-Armed Bandits (specifically UCB1 algorithm) to balance exploration and exploitation in algorithm selection.
- **AutoML Architecture Design:** Learning the software engineering principles required to integrate custom optimizers into existing hyperparameter search spaces.

Fundamentals Learned:

- **Machine Learning:** Developed our baseline understanding of model evaluation metrics and hyperparameter tuning principles, which are necessary to establish the Composite Scoring Formula.
- **Software Engineering & Distributed Systems:** Guided our architectural design modularity (creating independent scalable islands) and structured iterative execution planning.

5 Proposed Methodology

Core Architecture: Tiered Island Model with Bandit-Based Promotion

The system uses a population of N (randomly generated) individuals (hyperparameter configuration vectors) split into two tiers:

- **Tier 1 - Competition Arena (40%):** All algorithms (GA, PSO, DE, CMA-ES) operate on a batch of individuals (sharing the 40% individuals equally). They evolve independently and are scored every k iterations using a 5-metric composite score (Accuracy, Convergence Speed, Efficiency, Diversity, Performance Trend).
- **Tier 2 - Working Population (60%):** The top 2 algorithms from Tier 1 (by composite score) are promoted here as parallel islands (each getting half of the 60% individuals). Islands periodically share top 20% elite individuals with each other for cross-pollination.

Algorithm Switching: If a Tier-1 algorithm's composite score exceeds a Tier-2 algorithm's score, the Tier-1 algorithm is promoted and the Tier-2 incumbent is demoted. A grace period prevents instability and oscillating promotions.

Stagnation Handling (4 levels):

1. *T2 island stagnation:* Inject 20% random individuals; if persistent, UCB1 bandit selects replacement algorithm
2. *Single T1 algorithm stagnation:* Refresh bottom 50% of batch; temporarily shrink allocation

3. *Global T1 stagnation*: Keep top k% per island, replace rest with random individuals, re-warm-up
4. *Full population collapse*: Soft reset — keep top 30%, replace 70% with random fresh individuals

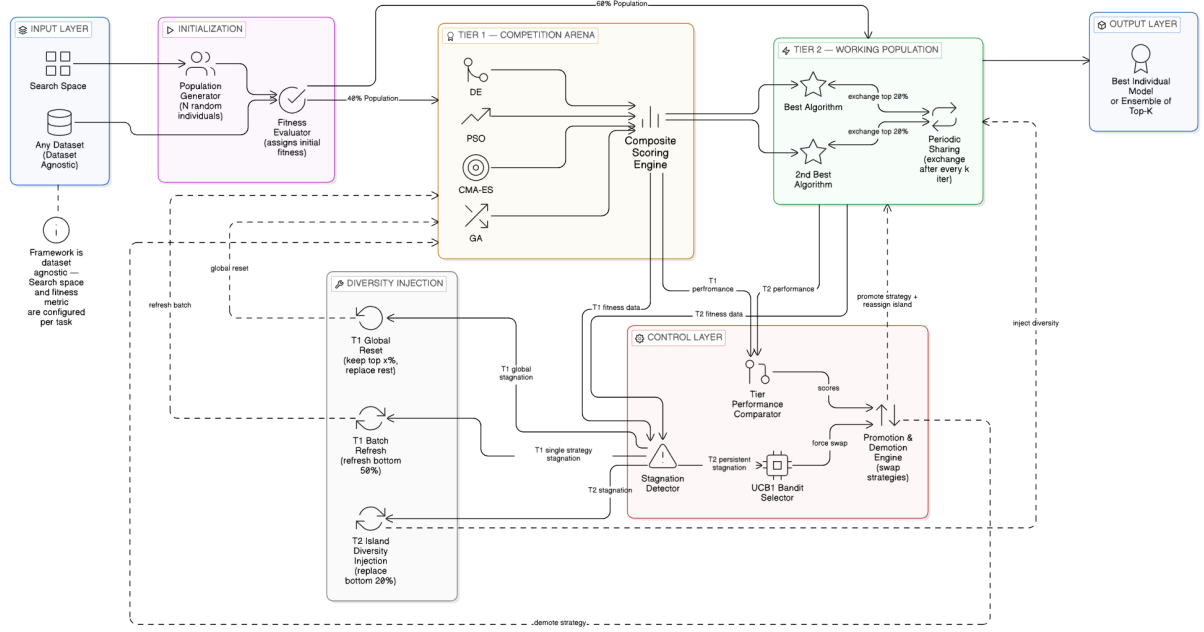


Figure 1: System Architecture: Dynamic Algorithm Switching with Tiered Islands

Composite Scoring Formula:

$$Score(S) = \sum_{i=1}^5 w_i \cdot \frac{M_i(S) - M_i^{\min}}{M_i^{\max} - M_i^{\min}}$$

where $w = [0.30, 0.25, 0.15, 0.15, 0.15]$ for Accuracy, Speed, Efficiency, Diversity, and Trend respectively. Min-max normalization is computed across all 4 algorithms.

UCB1 Bandit Algorithm Selection: For a stagnating Tier-2 island, the replacement algorithm a is chosen from the available pool to maximize its Upper Confidence Bound:

$$UCB1(a) = \hat{\mu}_a + C \sqrt{\frac{\ln(T)}{N_a}}$$

where $\hat{\mu}_a$ is the historical average composite score achieved by algorithm a , N_a is the number of times a has been deployed or evaluated, T is the total algorithmic selection events that have occurred, and C is the exploration-exploitation trade-off coefficient.

6 Current Status & Plan

Status: Literature survey completed; research gap identified; hybrid architecture designed; composite scoring and UCB1 bandit mechanisms formalized; complete workflow documented with edge cases and safeguards.

Next Steps:

- Implement Tier-1/Tier-2 population management module
- Implement composite scoring engine and stagnation detector
- Implement UCB1 bandit for algorithm selection
- Benchmark against single-algorithm baselines on CIFAR-10
- Validate dataset agnosticism on a second dataset (e.g., tabular)

References

- [1] E. L. Estevanell-Valladares, S. Estévez-Velarde, Y. Gutiérrez, A. Montoyo, and R. Mitkov, “XAutoLM: Efficient fine-tuning of language models via meta-learning and AutoML,” in *Proceedings of the Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2025, pp. 7863–7880.
- [2] G. Borboudakis, P. Charonyktakis, K. Paraschakis, and I. Tsamardinos, “A meta-level learning algorithm for sequential hyper-parameter space reduction in AutoML,” *arXiv preprint arXiv:2312.06305*, 2024.
- [3] M. Gerber, A. S. Bosman, and J. P. de Villiers, “Online meta-learning for AutoML in real-time,” *arXiv preprint arXiv:2502.20279*, 2025.
- [4] M. Crepinšek *et al.*, “Overcoming stagnation in metaheuristic algorithms with MsMA’s adaptive meta-level partitioning,” *Mathematics*, vol. 13, no. 11, p. 1803, 2025.
- [5] E. Alcobaça and A. C. P. L. F. de Carvalho, “A literature review on automated machine learning,” *Artificial Intelligence Review*, 2025.
- [6] A. Scrivano, “The emergence of automated machine learning (automl): Trends and future prospects,” *Research Square*, 2025, preprint.